

Practice Lab:

DSE Analytics, Word Count (first example), RDDs, Transforms, Actions, read and write from DSE

- This Practice Lab is dependent on Discussion Unit 7544, where most of the objects we create in this lab were introduced.
- This Practice Lab requires a working DSE system, with DSE Analytics enabled.
- Because of the dependency on the "dse spark" utility, this Practice Lab requires a ssh(C) prompt on at least one node operating DSE Analytics.
- *Artifacts created in this Practice Lab are used by other Practice Labs that follow.* So, it is important you complete the minimal challenges as outlined.

Challenge 1: Locate the Spark Master, state

from any DSE node (both commands)

`dse client-tool spark master-address`

`dse://127.0.0.1:9042?connection.local_dc=DC1;connection.host=;`



9042 ??

`dsetool ring`

Address	DC	Rack	Workload	Graph	Status	State	Load	Owns	Token	Health [0,1]
127.0.0.1	DC1	RAC1	Analytics(SM)	no	Up	Normal	116.01 KiB	?	-5915474479483571799	0.40



Note: you must specify a

keyspace to get ownership information.

SM == Spark Master

Challenge 2: Spark UIs

dse spark-sql

exit;

dse pyspark

quit()

dse sparkR # Will fail

- All of the REPLs are idiomatic, which you see from the exit commands
- Spark/SQL, generally ANSI SQL/99+, Hive/QL(Hive/SQL) 1.2.2; OLAP, not OLTP
- pyspark is a Python command interpreter
- sparkR will fail without given R language dependencies being met

Challenge 2: Spark UIs

dse spark

CONTROL-L // to clear screen

:quit



Stay in "dse spark", and **start a second subsequent session in another terminal window minutes later**

These are actually Scala interpreters; you don't need to run Spark

An 'action' submits a Spark job

Creating a new Spark Session

Spark context Web UI available at <http://172.16.119.194:4040>

Spark Context available as 'sc' (master = dse://?, app id = app-20180619165450-0001).

Spark Session available as 'spark'.

Spark SqlContext (Deprecated use Spark Session instead) available as 'sqlContext'

Challenge 2: http:// (Spark master IP) :7080

Running Applications

Application ID	Name	Workpool	Cores	Memory per Executor	Submitted Time	User	State	Duration
app-20180711090551-0004 (kill)	Spark shell	default	0	1024.0 MB	2018/07/11 09:05:51	anonymous	WAITING	23 s
app-20180711083439-0002 (kill)	Spark shell	default	3	1024.0 MB	2018/07/11 08:34:39	anonymous	RUNNING	32 min

Completed Applications

Application ID	Name	Workpool	Cores	Memory per Executor	Submitted Time	User	State	Duration
app-20180711084107-0003	PySparkShell	default	0	1024.0 MB	2018/07/11 08:41:07	anonymous	FINISHED	29 s
app-20180711083244-0001	SparkSQL::172.16.119.246	default	0	1024.0 MB	2018/07/11 08:32:44	anonymous	FINISHED	1.4 min

Challenge 2: dse spark, options

dse spark

--master spark://127.0.0.1:7080

--name NNN

--driver-memory 512M

--executor-memory 1G

--total-executor-cores N



Exit all but one
Spark REPL

About the REPLs:

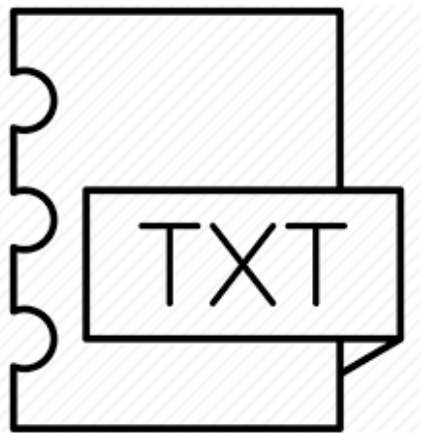
- Connects to Spark master, [Spark commands] run as jobs from a DAG, ..
- All rules apply; lazy evaluation, other
- SparkContext predefined as, sc
- SparkSession as, spark (SQL, Hive, other resources)
- Some cheats available; syntax not entirely same/rigid as a true program. (Similar to Gremlin development versus production.)

Challenge 3 (Optional):

Word Count (Hello World)

- Implement Word Count from Discussion Unit 7544
- As a simple 2-4 line task, we'll use the spark REPL as our IDE
- Easier to check classes, other.
- Read/write to/from DSE using RDDs.

Challenge 3 (Optional): Simple ASCII text file



Create a simple ASCII text file with data similar to,

111, Bob, Mary

222, Ted

333, Alice, Bob, Harold

444, Dave, Bob

Short/simple absolute pathname

```
val records = sc.textFile("file:///opt/stores_db/7545_HelloWorld.csv")
val words   = records.flatMap( record => record.split(",").drop(1) )
val counts1 = words.map( word => (word, 1) )
val counts2 = counts1.reduceByKey{ case(x, y) => x + y }

counts2.collect().foreach(println)
```

```
( Dave,1)
( Harold,1)
( Alice,1)
( Bob,3)
( Mary,1)
( Ted,1)
```



Challenge 3: The Scala, Spark code

Challenge 3: Two Transforms (Derivations)



```
val recs_withDerived = counts2.map  
  { case(name, cnt) =>  
    (name, cnt, name.toUpperCase ) }
```

```
object MyFunctions {  
  def generateFlag(arg: String) : String =  
  {  
    if (arg > "M")  
      "E-USA"  
    else  
      "W-USA"  
  }  
}  
  
val recs_withDerived = counts2.map  
{ case(name, cnt) =>  
  (name, cnt, name.toUpperCase,  
    MyFunctions.generateFlag(name) ) }  
recs_withDerived.collect().foreach(println  
)
```

Challenge 3: Two Transforms (Derivations)



Challenge 3: Read/Write to/from DSE

Use
`getClass()` to
prove this.



- At this point, you've likely created a 4 column RDD. Sample code is for 2 columns.
- Using CQLSH, create a similarly structured DSE/CQL table to read and write from.

Challenge 3: Write to DSE using RDDs

```
case class My_Record (pk: Int, value: String)
```

```
val my_records = sc.parallelize(Seq(  
  new My_Record(555, "Luis, Earl"),  
  new My_Record(666, "Ann, Sally, Bob"),  
  new My_Record(111, "Bob, Tony")  
))
```

```
my_records.saveToCassandra("ks_7545", "hello_world",  
  SomeColumns("pk", "value"))
```

```
// From CQLSH
```

```
SELECT * FROM ks_7545.hello_world ;
```

Challenge 3: Read from DSE using RDDs

```
val rows = sc.cassandraTable[My_Record]("ks_7545", "hello_world").  
  select("pk", "value").  
  as( (i:Int, s:String ) => new My_Record(i, s) )  
rows.collect().foreach(println)
```

Challenge 4:

Load customer, orders, items
DSE tables from CSV

- We need to load the customer, orders, and items tables from CSV; **we use these tables in later Practice Labs, and need these tables to exist and have data.**
- Some degree of busy work; load at least customer using Spark. You can cheat and use CQL/other for orders and items.
- Derive the DSE models from the SQL DDL. Do all DDL using CQLSH.
- Add two columns to customer to support the new, derived data.

Working Goal:

From: Maury_Atwater
To: DSE_HOTSHOT

Subject: Need this now !!!

Attached are 3 (count) SQL DDL files,
customer, orders, and items, and 3 (count)
ASCII Text CSV files.

I need these loaded into DSE now !!!

On customer.zipcode, derive an east coast,
west coast flag ("E-USA", "W-USA").
Also derive customer.company (company
name) to uppercase.

-MA



Maury Atwater, President
of Atwater's

The instructor will specify the location of (n) ASCII text files-

- SQL DDL for customer, order, items. Convert to CQL DDL; create whatever keyspace and tables.
- Load all 3 tables from CSV; use Spark/Scala for at least customer.
- Derive the 2 new columns for customer as specified on previous page.
- Don't worry about optimizing the data model. that work will come. Just do a 1:1 SQL -> CQL mapping.

Challenge 4: Load customer, order, items

Challenge 4: You are done when ..



- All three tables exist in DSE and have data; customer, orders, items
- You have at least the steps to complete customer in a nice (list)

Practice Lab:

Lessons learned





End of Unit: